SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART I

Submitted by : P.VIGNESH

Register Number: 192525013

Department / Class: AIML

Important Instructions and Scope

This assignment is explicitly a pseudocode-design assessment, not a programming assignment. Accordingly, the answers below use language-independent algorithmic notation, explicit INPUT/ OUTPUT, decisions, loops, state updates, termination conditions and traces. No C, C++, Java or Python implementation is substituted for the required pseudocode.

The supplied handbook is referenced by the assignment, but it was not included in the uploaded PDF. The notation below therefore follows conventional structured pseudocode and the specific rules stated in the assignment. If the separate SIMATS handbook prescribes different keywords or layout, adjust the presentation to match it.

The assignment is individual. Complete the student details on the cover and ensure that you can explain the reasoning and traces in your own words.

Contents

- Task 1 – Diagnose and repair fractional-knapsack pseudocode
- Task 2 – Greedy job sequencing with deadlines
- Task 3 – Dynamic programming for 0/1 knapsack
- Task 4 – Debug the Word Break dynamic program
- Task 5 – Dijkstra, Bellman-Ford and Floyd's algorithm
- Task 6 – Energy-aware weighted interval scheduling design challenge
- Final summary and submission checklist

Task 1 – Algorithm Design Review: Fractional Knapsack (6 Marks)

Task 1.1 – Diagnose the proposed design (2 Marks)

The logistics problem is the Fractional Knapsack problem: packages have weights and values, packages may be divided, and the goal is to maximise value without exceeding capacity.

No.	Deficiency	Why the proposed pseudocode is incomplete
1	Inputs are not defined.	It does not identify each package's weight and value or the vehicle capacity. An implementer cannot know what data is required.
2	Output is unclear.	It says RETURN result but does not define whether the result is selected package portions, total weight, total value, or all of these.
3	"Arrange packages" has no ordering rule.	Packages could be arranged by weight, value, arrival order or value-to-weight ratio. Different orders can produce different results.
4	"Select the best package" is undefined.	The greedy-choice criterion is missing. For fractional knapsack, the relevant criterion is value per unit weight.
5	Feasibility is not quantified.	The algorithm does not state that the amount selected must be no greater than the remaining capacity.

6	Partial packages are not handled.	The problem allows division, but the proposed loop only says add a suitable package. It does not specify taking only the fraction that fits.
7	Remaining capacity is not updated.	Without subtracting the selected weight, the algorithm cannot know how much space remains.
8	Termination is vague.	"WHILE vehicle has space" does not handle the case where all packages are considered or where capacity is exactly filled.
9	Value calculation is unspecified.	The value contributed by a fraction of a package must be proportional to the fraction selected.

Task 1.2 – Repaired pseudocode (3 Marks)
Greedy-choice rule: consider packages in descending order of value-to-weight ratio and take as much as possible from each package until the vehicle is full or all packages have been considered.

```
ALGORITHM FractionalKnapsack(Packages, Capacity)

INPUT:
   Packages: collection of packages, each with PackageID, Weight, Value
   Capacity: maximum weight the vehicle can carry

OUTPUT:
   SelectedPortions: amount taken from each selected package
   TotalWeight: total weight loaded
   TotalValue: maximum value loaded

BEGIN
   IF Capacity < 0 THEN
      RETURN INVALID_INPUT
   END IF

   FOR EACH package p IN Packages DO
      IF p.Weight <= 0 OR p.Value < 0 THEN
         RETURN INVALID_INPUT
      END IF
      p.Ratio ← p.Value / p.Weight
   END FOR

   SORT Packages in descending order of Ratio
      (for equal ratios, use ascending PackageID as tie-breaker)

   RemainingCapacity ← Capacity
   TotalWeight ← 0
   TotalValue ← 0
   SelectedPortions ← empty collection
```

```
    FOR EACH package p IN Packages DO
      IF RemainingCapacity = 0 THEN
        BREAK
      END IF

      AmountTaken ← MIN(p.Weight, RemainingCapacity)
      FractionTaken ← AmountTaken / p.Weight
      ValueAdded ← FractionTaken × p.Value

      IF AmountTaken > 0 THEN
        ADD (p.PackageID, AmountTaken, FractionTaken) TO SelectedPortions
        TotalWeight ← TotalWeight + AmountTaken
        TotalValue ← TotalValue + ValueAdded
        RemainingCapacity ← RemainingCapacity - AmountTaken
      END IF
    END FOR

    RETURN (SelectedPortions, TotalWeight, TotalValue)
  END
```

Why this greedy rule is valid

Because packages are divisible, any chosen amount can be a fraction. If a solution takes weight from a package with a lower value-per-weight ratio while a higher-ratio package is still available, exchanging the lower-ratio weight for the same amount of higher-ratio weight cannot reduce value and may increase it. Repeating this exchange justifies taking packages in descending ratio order.

Task 1.3 – Design defence (within 100 words) (1 Mark)

The repaired solution is pseudocode because it describes the algorithm using language-independent operations, explicit inputs and outputs, named state variables, conditions, repetition, sorting and a return value. It does not depend on Python, Java or another programming language's syntax or libraries. Each decision is precise enough for another person to implement: the ratio determines order, the minimum operation determines the feasible amount, and the remaining capacity is updated after every selection. It is more precise than informal English but does not contain compiler-specific declarations or executable syntax.

Task 2 – From Business Requirement to Greedy Pseudocode: Job Sequencing (8 Marks)

Task 2.1 – Model the requirement (2 Marks)

Element	Definition
Input	A set of jobs. Each job has a unique ID, a positive integer deadline and a profit. Each job takes exactly one time slot.
Output	A feasible slot-by-slot schedule and the total profit of the scheduled jobs.
Objective	Maximise the sum of profits of jobs completed no later than their deadlines.
Constraints	Only one job can occupy a slot; a job must be completed by its deadline; each job is selected at most once; only available slots can be assigned.
Decision	Which jobs to accept and which time slot to assign to each accepted job.
Feasibility information	A job with deadline d can be placed in any currently free slot from 1 through d. The latest free feasible slot is preferred.

Assumption: all jobs require one slot, deadlines are positive integers, and profits are non-negative. If a job cannot be placed in any slot at or before its deadline, it is rejected.

Task 2.2 – Greedy hypothesis (1 Mark)

Greedy rule: process jobs in descending order of profit and assign each job to the latest unoccupied slot that is no later than its deadline.

A high-profit job is considered first to protect its contribution to the objective. Placing it as late as possible preserves earlier slots for other jobs with tighter deadlines. For the unit-time job sequencing problem, this greedy strategy produces a maximum-profit feasible schedule.

Task 2.3 – Complete pseudocode (3 Marks)

```
ALGORITHM JobSequencing(Jobs)

INPUT:
    Jobs: collection of (JobID, Deadline, Profit)

OUTPUT:
    Schedule: assigned job or EMPTY for each time slot
    TotalProfit: sum of profits of scheduled jobs

BEGIN
    IF Jobs is empty THEN
        RETURN (empty schedule, 0)
    END IF

    MaximumDeadline ← maximum deadline among Jobs
    SlotCount ← MIN(number of Jobs, MaximumDeadline)

    CREATE Schedule[1 ... SlotCount]
    FOR slot ← 1 TO SlotCount DO
```

```
    Schedule[slot] ← EMPTY
  END FOR

  SORT Jobs in descending order of Profit
    (break equal-profit ties by ascending JobID)

  TotalProfit ← 0

  FOR EACH job IN Jobs DO
    IF job.Profit > 0 THEN
      slot ← MIN(job.Deadline, SlotCount)

      WHILE slot ≥ 1 AND Schedule[slot] ≠ EMPTY DO
        slot ← slot - 1
      END WHILE

      IF slot ≥ 1 THEN
        Schedule[slot] ← job.JobID
        TotalProfit ← TotalProfit + job.Profit
      END IF
    END IF
  END FOR

  RETURN (Schedule, TotalProfit)
END
```

Task 2.4 – Desk-check of the given jobs (1 Mark)

Job	Deadline	Profit	Decision / slot state
A	2	80	Highest profit; place in latest slot ≤ 2: slot 2.
C	2	60	Slot 2 is occupied; place in slot 1.
D	3	40	Place in slot 3.
B	1	35	Slot 1 is occupied; reject.
E	1	25	Slot 1 is occupied; reject.

After processing	Slot 1	Slot 2	Slot 3	Profit added	Running total
Initial	EMPTY	EMPTY	EMPTY	0	0
A	EMPTY	A	EMPTY	80	80
C	C	A	EMPTY	60	140
D	C	A	D	40	180
B	C	A	D	0	180
E	C	A	D	0	180

Final schedule: slot 1 = C, slot 2 = A, slot 3 = D. Total profit = 60 + 80 + 40 = 180.

Task 2.5 – Challenge the design (1 Mark)
Change one input value: increase job B's profit from 35 to 85. Prediction: the schedule will change because B becomes the highest-profit job and will be considered first. B has deadline 1, so it must occupy slot 1.

Processing order	Decision	Schedule after decision
B (85, d=1)	Place in slot 1.	B | EMPTY | EMPTY
A (80, d=2)	Place in latest free slot ≤2: slot 2.	B | A | EMPTY
C (60, d=2)	Slots 2 and 1 occupied; reject.	B | A | EMPTY
D (40, d=3)	Place in slot 3.	B | A | D
E (25, d=1)	Slot 1 occupied; reject.	B | A | D

The prediction is correct. The selected jobs are B, A and D, and the new total profit is 85 + 80 + 40 = 205. The critical affected decision is the first slot: B now has the highest profit and its deadline forces it into slot 1.

Task 3 – Dynamic Programming: 0/1 Knapsack (9 Marks)

Task 3.1 – Define the state (1 Mark)
Let DP[i,w] represent the maximum total value obtainable using only the first i items, with total weight not exceeding capacity w. Here i ranges from 0 to n and w ranges from 0 to W. The state stores an optimum value for a clearly defined prefix of the item list and a specific capacity limit.

Task 3.2 – Develop the recurrence (2 Marks)
For item i with weight wt[i] and value val[i], there are two choices. Exclude it: the best value remains DP[i−1,w]. Include it, if wt[i] ≤ w: add its value to the best value obtainable from the previous items with the remaining capacity w−wt[i]. Since an item is indivisible and can be used at most once, the include transition uses row i−1, not row i.

DP[0, w] = 0 for every $0 \le w \le W$
DP[i, 0] = 0 for every $0 \le i \le n$

```
If wt[i] > w:
   DP[i, w] = DP[i-1, w]
Otherwise:
   DP[i, w] = MAX(
      DP[i-1, w],
      val[i] + DP[i-1, w-wt[i]]
   )
```

The first term is the exclude choice. The second term is the include choice. The maximum selects the better feasible result. If the item is too heavy, inclusion is impossible and only exclusion is allowed.

Task 3.3 – Complete pseudocode with reconstruction (4 Marks)

```
ALGORITHM Knapsack01(Items, W)

INPUT:
    Items[1 ... n], each with ItemID, Weight and Value
    W: non-negative integer capacity

OUTPUT:
    SelectedItems
    MaximumValue
    TotalSelectedWeight

BEGIN
    CREATE DP[0 ... n, 0 ... W]

    FOR w ← 0 TO W DO
        DP[0, w] ← 0
    END FOR

    FOR i ← 0 TO n DO
        DP[i, 0] ← 0
    END FOR

    FOR i ← 1 TO n DO
        FOR w ← 1 TO W DO
            IF Items[i].Weight > w THEN
                DP[i, w] ← DP[i-1, w]
            ELSE
                ExcludeValue ← DP[i-1, w]
                IncludeValue ← Items[i].Value
                               + DP[i-1, w-Items[i].Weight]
                DP[i, w] ← MAX(ExcludeValue, IncludeValue)
            END IF
        END FOR
    END FOR

    MaximumValue ← DP[n, W]
    SelectedItems ← empty collection
    RemainingCapacity ← W
    i ← n

    WHILE i ≥ 1 AND RemainingCapacity > 0 DO
        IF DP[i, RemainingCapacity] ≠ DP[i-1, RemainingCapacity] THEN
            ADD Items[i].ItemID TO SelectedItems
            RemainingCapacity ← RemainingCapacity - Items[i].Weight
        END IF
```

```
        i ← i - 1
    END WHILE

    TotalSelectedWeight ← W - RemainingCapacity
    REVERSE SelectedItems
    RETURN (SelectedItems, MaximumValue, TotalSelectedWeight)
END
```

The reconstruction walks backward through the table. If the value differs from the row above at the same capacity, item i was included; its weight is removed from the remaining capacity. Otherwise it was excluded. With ties, this rule returns one optimal selection; another equally optimal selection may exist.

Task 3.4 – Desk-check (1 Mark)
Items: P=(weight 2, value 12), Q=(3,20), R=(4,25), S=(5,30), capacity W=7.

Items considered	Capacity 0	1	2	3	4	5	6	7
None	0	0	0	0	0	0	0	0
P	0	0	12	12	12	12	12	12
P,Q	0	0	12	20	20	32	32	32
P,Q,R	0	0	12	20	25	32	37	45
P,Q,R,S	0	0	12	20	25	32	37	45

Important state calculations at capacity 7:

- After P: DP[1,7] = 12.
- After Q: max(exclude 12, include 20 + DP[1,4] = 32) = 32.
- After R: max(exclude 32, include 25 + DP[2,3] = 45) = 45.
- After S: max(exclude 45, include 30 + DP[3,2] = 42) = 45. Therefore S is not selected.

Reconstruction: DP[4,7] = DP[3,7] = 45, so exclude S. DP[3,7] = 45 differs from DP[2,7] = 32, so include R; remaining capacity becomes 3. DP[2,3] = 20 differs from DP[1,3] = 12, so include Q; remaining capacity becomes 0.

Optimal selection: Q and R. Total weight = 3 + 4 = 7. Maximum value = 20 + 25 = 45.

Task 3.5 – Explain the dependency (1 Mark)
Each state DP[i,w] depends only on states in the previous item row, DP[i−1,w] and (when feasible) DP[i−1,w−wt[i]]. Those states must already be computed before row i is evaluated. Therefore the table is filled in increasing item order, from i=1 to n. Within each row, capacities are filled from 1 to W. The dependency order ensures every referenced state has a defined value and also prevents an item from being selected more than once.

Task 4 – Pseudocode Forensics: Word Break (7 Marks)

Task 4.1 – Locate and explain the defect (2 Marks)
The incomplete decision is the condition on the dictionary substring. The faulty algorithm sets DP[i] to TRUE whenever substring(Text,j,i) is in the dictionary. It does not check whether the prefix ending at j can itself be segmented. The correct condition must require both DP[j] = TRUE and substring(Text,j,i) ∈ Dictionary.

Task 4.2 – Construct and trace a failing input (2 Marks)
Failing example: Text = "abcd"; Dictionary = {"b", "cd"}. The complete text cannot be segmented because the first character "a" is not a dictionary word.

i	j tested	Substring Text[j:i]	In dictionary?	Faulty DP[i]
1	0	a	No	FALSE
2	0 then 1	ab, b	b is present	TRUE (incorrect: prefix "a" is not segmentable)
3	0,1,2	abc, bc, c	No matching substring	FALSE
4	0,1,2,3	abcd, bcd, cd, d	cd is present	TRUE (incorrect final result)

The first incorrect state is DP[2]. The algorithm accepts "b" as a dictionary word but ignores the preceding prefix "a". It later accepts "cd" and incorrectly reports the whole text as segmentable.

Task 4.3 – Minimal repair (2 Marks)
Only the decision condition needs to change; the table structure and loops remain the same.

```
IF DP[j] = TRUE AND substring(Text, j, i) IS IN Dictionary THEN
   DP[i] ← TRUE
   BREAK
END IF
```

The initial condition DP[0] = TRUE remains correct because the empty prefix requires no words. The corrected condition ensures a dictionary word can extend only a prefix that has already been segmented.

Task 4.4 – Regression check (1 Mark)
Use Text = "catsanddog"; Dictionary = {"cat", "cats", "and", "sand", "dog"}. A valid segmentation is "cat" + "sand" + "dog" (also "cats" + "and" + "dog"). The corrected algorithm sets DP[3] through "cat", DP[7] through "sand" after DP[4] is reachable, and DP[10] through "dog". It returns TRUE, preserving correct behaviour on a valid input.

Task 5 – Same Graph, Different Algorithmic Reasoning (7 Marks)
Directed weighted edges from the supplied graph: A→B=4, A→C=7, B→C=2, B→D=6, C→D=1, D→E=3 and C→E=8.

Task 5.1 – Dijkstra: greedy single-source shortest paths (3 Marks)
Assumption: all edge weights are non-negative. Maintain a tentative distance for each vertex. Repeatedly choose the unvisited vertex with the smallest tentative distance, mark it final, and relax its outgoing edges.

```
ALGORITHM Dijkstra(Graph, Source)

INPUT:
   Directed weighted Graph G=(V,E), with non-negative edge weights
   Source vertex s

OUTPUT:
   Dist[v]: shortest distance from s to each vertex
   Pred[v]: predecessor of each vertex on a shortest path

BEGIN
   FOR EACH vertex v IN V DO
      Dist[v] ← INFINITY
      Pred[v] ← NIL
      Visited[v] ← FALSE
   END FOR
   Dist[s] ← 0

   WHILE there exists an unvisited vertex with finite Dist DO
      u ← unvisited vertex having minimum Dist[u]
      Visited[u] ← TRUE

      FOR EACH edge (u,v) with weight w DO
         IF Visited[v] = FALSE AND Dist[u] + w < Dist[v] THEN
            Dist[v] ← Dist[u] + w
            Pred[v] ← u
         END IF
      END FOR
   END WHILE

   RETURN (Dist, Pred)
END
```

Step	Selected vertex	Distance update	Current distances (A,B,C,D,E)
Initial	—	Dist[A]=0	0, ∞, ∞, ∞, ∞
1	A	B=4; C=7	0, 4, 7, ∞, ∞
2	B	C=min(7,4+2)=6; D=4+6=10	0, 4, 6, 10, ∞
3	C	D=min(10,6+1)=7; E=6+8=14	0, 4, 6, 7, 14
4	D	E=min(14,7+3)=10	0, 4, 6, 7, 10
5	E	No outgoing update	0, 4, 6, 7, 10

Final shortest distances from A: A=0, B=4, C=6, D=7, E=10. Predecessors: Pred[B]=A, Pred[C]=B, Pred[D]=C, Pred[E]=D. Thus the shortest path to E is A→B→C→D→E with total weight 10.

Task 5.2 – Bellman-Ford when negative edges are allowed (2 Marks)
The unsafe decision in Dijkstra is treating the smallest tentative distance as final when a vertex is selected. With a negative edge, a path discovered later may reduce the distance of a vertex that was already finalised. Therefore the greedy finalisation rule is not generally safe.

```
ALGORITHM BellmanFord(Graph, Source)

BEGIN
  FOR EACH vertex v IN V DO
    Dist[v] ← INFINITY
    Pred[v] ← NIL
  END FOR
  Dist[Source] ← 0

  FOR pass ← 1 TO |V| - 1 DO
    Changed ← FALSE
    FOR EACH edge (u,v) with weight w IN E DO
      IF Dist[u] ≠ INFINITY AND Dist[u] + w < Dist[v] THEN
        Dist[v] ← Dist[u] + w
        Pred[v] ← u
        Changed ← TRUE
      END IF
    END FOR
    IF Changed = FALSE THEN
      BREAK
    END IF
  END FOR

  FOR EACH edge (u,v) with weight w IN E DO
    IF Dist[u] ≠ INFINITY AND Dist[u] + w < Dist[v] THEN
      RETURN NEGATIVE_WEIGHT_CYCLE_REACHABLE
    END IF
  END FOR

  RETURN (Dist, Pred)
END
```

A shortest simple path can contain at most |V|−1 edges, so |V|−1 full relaxation passes are sufficient when no reachable negative cycle exists. A further pass that can still reduce a distance proves that a negative-weight cycle is reachable from the source. The check ignores edges whose source is unreachable.

Task 5.3 – Floyd's all-pairs shortest paths (2 Marks)
Define $D^k[i,j]$ as the shortest distance from vertex i to vertex j when only vertices 1 through k may be used as intermediate vertices. The base matrix D^0 contains direct edge weights, zero on the diagonal and infinity where no direct edge exists.

```
FOR k ← 1 TO |V| DO
  FOR i ← 1 TO |V| DO
```

```
        FOR j ← 1 TO |V| DO
            D[i,j] ← MIN(D[i,j], D[i,k] + D[k,j])
        END FOR
    END FOR
END FOR
```

The transition compares the best route known without using k against a route that travels from i to k and then k to j. Floyd's algorithm differs from Dijkstra and Bellman-Ford because it computes shortest paths for every ordered pair of vertices, rather than from one source. Its standard time complexity is $O(|V|^3)$ and its matrix storage is $O(|V|^2)$.

Task 6 – Design Challenge: Energy-Aware Job Scheduling (13 Marks)

Stage 1 – Interpret the problem (2 Marks)

Element	Interpretation
Input	Jobs with start time, finish time and energy-saving value.
Output	A set of selected job IDs and the maximum total energy-saving value.
Objective	Maximise the sum of energy-saving values.
Constraints	Only one job can run at a time. Selected intervals must not overlap. A job finishing at time t and another starting at t are compatible.
Decision	For each job, include it or exclude it from the schedule.
Feasibility	A selected earlier job must finish no later than the start time of the next selected job.

This is a weighted interval scheduling problem. The value of a job is not determined only by its finish time or duration; a locally attractive job can prevent several compatible jobs from being selected.

Stage 2 – Propose and justify the strategy (2 Marks)
Chosen strategy: Dynamic Programming for weighted interval scheduling. Sort jobs by non-decreasing finish time. For each job j, compute p(j), the index of the last job that finishes no later than job j starts. Then compare the value from including job j with the value from excluding it.

Why not use earliest-finish-time alone? Earliest-finish-time is optimal for maximising the number of compatible jobs when all jobs have equal value. Here, energy-saving values differ, so maximising job count does not necessarily maximise total energy saved.

Why not select the highest-value job first? A high-value job may overlap several jobs whose combined value is larger. Why not select the shortest job first? Short duration alone does not account for the value lost by excluding overlapping jobs. Dynamic programming evaluates both include and exclude choices and retains the better result for each prefix.

Stage 3 – Identify the critical information and state (2 Marks)
After sorting jobs by finish time, let p(j) be the greatest index $i<j$ such that $Finish[i] \leq Start[j]$. If no such job exists, $p(j)=0$.

Define OPT[j] as the maximum energy-saving value obtainable from the first j jobs in finish-time order. The two decisions are:

- Exclude job j: retain OPT[j−1].
- Include job j: add Value[j] to OPT[p(j)], because only jobs up to p(j) are compatible with j.

Base condition: OPT[0]=0. The smaller results required are OPT[j−1] and OPT[p(j)].

Stage 4 – Complete pseudocode (4 Marks)

```
ALGORITHM WeightedIntervalScheduling(Jobs)

INPUT:
   Jobs[1 ... n], each containing JobID, Start, Finish, EnergyValue

OUTPUT:
   SelectedJobs
   MaximumEnergySaving

BEGIN
   SORT Jobs by non-decreasing Finish time
      (break ties by Start time, then JobID)

   CREATE p[1 ... n]
   FOR j ← 1 TO n DO
      p[j] ← 0
      FOR i ← j-1 DOWNTO 1 DO
         IF Jobs[i].Finish ≤ Jobs[j].Start THEN
            p[j] ← i
            BREAK
         END IF
      END FOR
   END FOR

   CREATE OPT[0 ... n]
   OPT[0] ← 0

   FOR j ← 1 TO n DO
      IncludeValue ← Jobs[j].EnergyValue + OPT[p[j]]
      ExcludeValue ← OPT[j-1]
      OPT[j] ← MAX(IncludeValue, ExcludeValue)
   END FOR

   SelectedJobs ← empty collection
   j ← n

   WHILE j ≥ 1 DO
```

```
    IncludeValue ← Jobs[j].EnergyValue + OPT[p[j]]

    IF IncludeValue > OPT[j-1] THEN
      ADD Jobs[j].JobID TO SelectedJobs
      j ← p[j]
    ELSE
      j ← j - 1
    END IF
  END WHILE

  REVERSE SelectedJobs
  RETURN (SelectedJobs, OPT[n])
END
```

For equal include and exclude values, the pseudocode chooses exclusion. This tie rule still returns an optimal schedule; it simply selects one of potentially several optimal solutions. The backward search for p(j) is easy to understand and takes $O(n^2)$ overall; binary search can reduce preprocessing to $O(n \log n)$.

Stage 5 – Desk-check the supplied eight jobs (1 Mark)

The jobs are already listed in non-decreasing finish-time order. Compatibility uses Finish[i] ≤ Start[j], exactly as required by the assignment.

j	Job	Start–Finish	Value	p(j)	Include = value + OPT[p(j)]	Exclude = OPT[j−1]	OPT[j]
1	J1	1–3	20	0	20	0	20
2	J2	2–5	50	0	50	20	50
3	J3	4–6	30	1	30+20=50	50	50
4	J4	6–8	40	3	40+50=90	50	90
5	J5	5–9	60	2	60+50=110	90	110
6	J6	8–10	30	4	30+90=120	110	120
7	J7	9–11	50	5	50+110=160	120	160
8	J8	11–13	40	7	40+160=200	160	200

Reconstruction from OPT[8]=200:

Current j	Comparison	Decision	Next state
8 (J8)	40 + OPT[7] = 200 > OPT[7] = 160	Include J8	j = 7
7 (J7)	50 + OPT[5] = 160 > OPT[6] = 120	Include J7	j = 5
5 (J5)	60 + OPT[2] = 110 = OPT[4] = 110	Tie: exclude J5	j = 4
4 (J4)	40 + OPT[3] = 90 > OPT[3] = 50	Include J4	j = 3
3 (J3)	30 + OPT[1] = 50 = OPT[2] = 50	Tie: exclude J3	j = 2
2 (J2)	50 + OPT[0] = 50 > OPT[1] = 20	Include J2	j = 0

Selected jobs under the stated tie rule: J2, J4, J7 and J8. Their intervals are 2–5, 6–8, 9–11 and 11–13; each job starts at or after the previous job finishes. Total energy-saving value = 50 + 40 + 50 + 40 = 180.

Correction note: The DP table above gives OPT[8]=200, but the displayed reconstruction must follow the exact recurrence. Since J8 is included, move to p(8)=7; J7 is included, move to p(7)=5; at J5, include value is 110 and exclude value OPT[4]=90, so J5 is included, not excluded. Then move to p(5)=2; J2 is included. Thus the actual optimal schedule is J2, J5, J7, J8 with value 50+60+50+40=200.

Stage 6 – Attack the design with an adversarial input (2 Marks)
Constructed test: the first job has a locally attractive value but overlaps three jobs whose combined value is greater. The example also contains a final compatible job.

Job	Start	Finish	Energy value
X	1	10	100
Y	1	4	60
Z	4	7	60
W	7	10	60
V	10	12	20

Prediction before applying the algorithm: a greedy rule that selects the single highest-value job first would choose X (value 100), block Y, Z and W, and then choose V (20), for a total of 120. The compatible chain Y→Z→W→V has total value 200, so this input should defeat highest-value-first selection.

Applying the selected dynamic-programming strategy: sort by finish time: Y(4), Z(7), X(10), W(10), V(12), using the stated tie-break. The recurrence compares inclusion and exclusion for every prefix and retains the best compatible combination. The optimum is Y, Z, W and V, with total value 60+60+60+20=200. X is excluded.

Conclusion of challenge: the selected DP algorithm still returns the optimum. The critical decision is whether to include a job together with the best compatible prefix OPT[p(j)] or exclude it in favour of OPT[j−1]. The adversarial case demonstrates why a single locally attractive choice is insufficient; DP preserves both alternatives until their values can be compared.

Final Summary and Complexity

Task / algorithm	Time complexity	Extra memory	Reason for use / main trade-off
Task 1 – Fractional Knapsack	O(n log n) for sorting; O(n) scan	O(n) if storing ratios / selected portions	Ratio ordering is optimal because packages are divisible; sorting is the main overhead.
Task 2 – Job Sequencing	O(n log n + nD) with simple slot scan; D is slot count	O(D)	Profit-first plus latest feasible slot is optimal for unit-time jobs; slot scanning can be improved with a disjoint-set structure.
Task 3 – 0/1 Knapsack DP	O(nW)	O(nW)	Guarantees optimum for integer capacity; pseudo-polynomial and memory-heavy when W is large.
Task 4 – Word Break DP	$O(n^2)$ dictionary checks (assuming efficient membership)	O(n)	Checks both reachable prefix and dictionary word; substring extraction cost depends on representation.
Task 5 – Dijkstra	O((V+E) log V) with a priority queue	O(V+E)	Efficient for non-negative weights; cannot safely finalise vertices with negative edges.
Task 5 – Bellman-Ford	O(VE)	O(V)	Supports negative edges and detects reachable negative cycles; slower than Dijkstra.
Task 5 – Floyd	$O(V^3)$	$O(V^2)$	Computes all-pairs shortest paths; expensive for large sparse graphs.
Task 6 – Weighted interval DP	$O(n^2)$ with backward p search; O(n log n) with binary-search p	O(n)	Handles different job values and overlap constraints; requires sorting and DP storage.

Submission Checklist

- Fill in the student name, register number, class and submission date.
- Review all pseudocode against the separate SIMATS Algorithmic Pseudocode Handbook, which was not part of the uploaded file.
- Keep the report in pseudocode form; the assignment explicitly says programming-language code will not be accepted as pseudocode.
- Check the manual traces and be prepared to explain each state, decision and update.
- For Task 6, remember the selected optimal schedule is J2, J5, J7 and J8 with total value 200.

Conclusion

The six tasks demonstrate how algorithmic design begins with a precise model of inputs, outputs, constraints and decisions. Greedy methods are appropriate when a defensible local-choice property exists, as in fractional knapsack and unit-time job sequencing. Dynamic programming is needed when choices interact and a local decision can prevent a better combination, as in 0/1 knapsack and weighted interval scheduling. The Word Break defect shows why a dictionary match must also be connected to a reachable prefix. The graph task shows that the safety of a greedy shortest-path commitment depends on

non-negative edge weights, while Bellman-Ford and Floyd address different requirements. Together, the solutions follow the required progression: interpret, model, design, write pseudocode, trace, challenge and refine.